

数理最適化研修 報告

—— LP / IP / MIP を学んで MaxCut の解き方を整理する ——

原典: BrainPad『数理最適化研修』全 105 ページ

発表者: 五十嵐研 CIM-CAC-MaxCut チーム

本日のアジェンダ

20 枚で 105 ページを縮約

- Part I 数理最適化とは — 第 1～4 枚
 - 最適化問題の 3 要素と分類
 - 解くまでの流れ・ソルバーの位置づけ
- Part II LP / IP / MIP — 第 5～13 枚
 - 線形計画 (LP): 飲料生産問題
 - 整数計画 (IP, 0-1): ナップサック
 - 混合整数計画 (MIP): 施設配置・チーム分け
- Part III 研究室との接続 — 第 14～18 枚
 - MaxCut を IP として書く
 - MILP ソルバー vs CIM/CAC の住み分け
 - Optuna も「最適化問題」だった
- まとめ・参考文献 — 第 19～20 枚

本資料のゴール

- BrainPad の研修内容(LP/IP/MIP)を、研究室で
- 自分たちが解いている MaxCut / CIM とどう繋がるか
- という観点に置き換えて整理する。
-
- テクニックそのものではなく、
- 「自分たちの問題を数理最適化の言葉で
- 記述・分類する力」を持ち帰る。
-
- → MaxCut は 0-1 IP の典型問題
- → CIM はフルスクラッチ系アルゴリズム
- → Optuna は連続ブラックボックス最適化

数理最適化とは何か

①数理最適化イントロダクション

制約条件のもとで目的関数を最大化/最小化する解を求める「問題」と、それを通じた意思決定の「技術」

決定変数

- 最適化で決めたい値
- $x \in \mathbb{R}^n$ (連続) や $\{0,1\}^n$ (離散)

目的関数 $f(x)$

- 最大化 / 最小化したい指標
- 売上、損失、コスト、...

制約条件 $g(x) \leq 0$

- 実行可能領域を絞る式
- 容量、需要、論理関係、...

機械学習(予測) ↔ 数理最適化(意思決定)

- ML: 「次にどうなる？」を入力データから当てるモデル → 説明変数 ⇒ 目的変数
- MO: 「どうしたら良い？」を目的関数の最大化で答える → 数理モデル ⇒ 行動
- 本研究室の CIM 研究も「ML がやらないこと」を扱う立場 — 意思決定 (=どう切るか) に近い

最適化問題の分類

用語と地図

決定変数と目的・制約が「線形か / 連続か」で大別される

連続 × 線形

- LP
Linear Programming
-
- 実数決定変数 + 線形 f, g
例: 生産量配分

連続 × 非線形

- NLP / QP
-
- 実数決定変数 + 非線形
例: ML の損失関数最小化
本研究室の CIM パラ調整

離散 × 線形

- IP / MIP
-
- 整数 (0/1) 決定変数 + 線形
例: ナップサック、施設配置
★MaxCut もここ

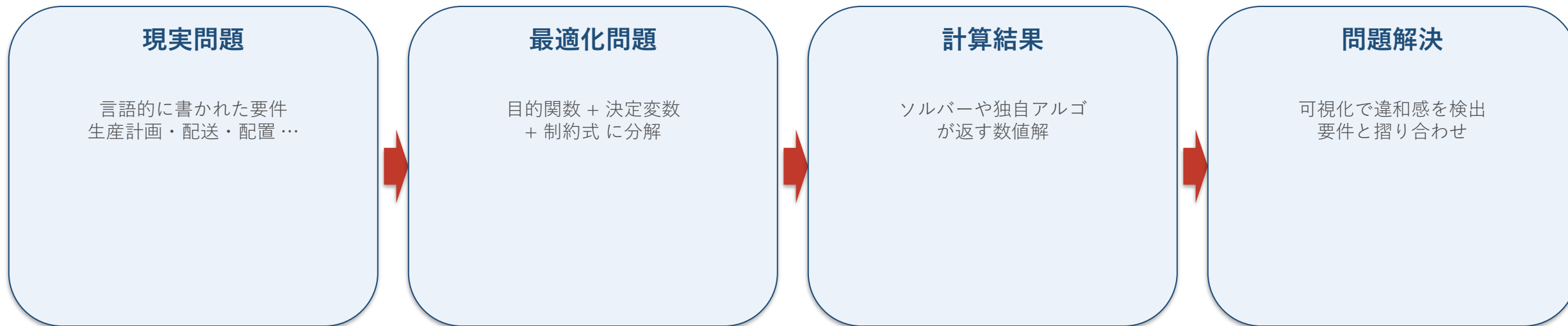
離散 × 非線形

- INLP / QUBO
-
- 整数決定変数 + 非線形
例: QUBO, Ising モデル
★CIM が解いているのは実質これ

★ = 自分たちの研究テーマがここに位置することを示す

最適化案件の進め方

定式化 → 求解 → 可視化 → 改良



↑ 1回でうまく行くことは稀。再定式化 / 再アルゴリズム選択を反復する

- 研究室文脈の翻訳:
 - 「現実問題」=最大カット数を取りたい / 振幅が綺麗に二極化したい
 - 「定式化」=MaxCut → 0-1 IP / Ising → SDE
 - 「ソルバー」=Cbc / Gurobi / SCIP vs CIM / CAC / SA / SB
 - 「可視化・改良」=cut 分布ヒスト・振幅軌跡 → パラメータ再調整

汎用ソルバー vs フルスクラッチ

性能と汎用性のトレードオフ

(1) 汎用ソルバー

- 予めアルゴリズムを実装したパッケージ
- Gurobi (有償) / SCIP / Cbc / CP-SAT
-
- ○ 制約変更に強い・保守容易
- ○ 「裏で何をやっているか」を意識せず使える
- × 問題固有の構造は使えない
- × 大規模になると現実時間で解けない
-
- 案件ではこれが第一選択
- (BrainPad 研修も PuLP + Cbc 想定)

(2) フルスクラッチ・問題固有アルゴリズム

- 自分でアルゴリズムを 1 から実装
- ヒューリスティクス / メタヒューリスティクス
-
- ○ 問題固有の性質を最大活用 → 高速・大規模対応
- × 保守性が低い・汎用性なし
- × 設計・チューニング・検証コストが高い
-
- ★研究室の CIM / CAC / SB / SA はここに該当
- 汎用 MILP ソルバーが大規模 G-set を解けない領域で、
- Ising 物理を使って高速に近似解を得る

Part II

LP / IP / MIP の定式化

線形計画問題 (LP)

② 線形計画 定式化

いくつかの一次不等式/等式を制約として、ある一次関数を最大化・最小化する問題

基本性質

- 最大化と最小化は等価
- $\max f(x) \Leftrightarrow \min -f(x)$
-
- 不等式制約と等式制約も等価
- $ax \leq b \Leftrightarrow ax + s = b, s \geq 0$
-
- 実行可能領域は多面体になる
- 最適解は通常その「頂点」に位置 (単体法の根拠)
-
- ソルバーは多項式時間で解ける
- $\rightarrow$ LP は事実上「解ける問題」

一般形

- minimize $c^T x$
- subject to $Ax \leq b$
- $x \geq 0$
-
- $c \in \mathbb{R}^n$ 目的関数の重み
- $A \in \mathbb{R}^{m \times n}$ 制約行列
- $b \in \mathbb{R}^m$ 制約の右辺
- $x \in \mathbb{R}^n$ 決定変数 (実数!)
-
- 決定変数が実数なのが LP の本質
- 整数化されると一気に難しくなる $\Rightarrow$ 次節 IP

LP 例題: 飲料の生産量最適化

定式化の 3 ステップを体験

問題設定

- 売値: フルーツジュース 3,000 円/kg, ティー 2,000 円/kg
- 材料: リンゴ・オレンジ(在庫 500 / 300)
- 1 kg 製造あたり必要量:
- ジュース → リンゴ 2, オレンジ 4
- ティー → リンゴ 5, オレンジ 2
- 売上を最大化するには何 kg ずつ作る?
- 「現実問題」をどうやって
- 決定変数 / 目的関数 / 制約 に分解するか
- がこの章で持ち帰るべきスキル

定式化

- 決定変数: $x_1 = \text{ジュース量}, x_2 = \text{ティー量} \text{ (kg, } \geq 0 \text{)}$
- 目的関数:
- maximize $3000 x_1 + 2000 x_2$
- 制約条件:
- リンゴ: $2 x_1 + 5 x_2 \leq 500$
- オレンジ: $4 x_1 + 2 x_2 \leq 300$
- 非負: $x_1, x_2 \geq 0$
- 解: $x_1 = 31.25, x_2 = 87.5,$
- 売上 = 268,750 円

PuLP + Cbc による実装

③ 線形計画 実装

モデラー (PuLP) で定式化を Python で書き、ソルバー (Cbc) が裏で解く

```
import pulp

model = pulp.LpProblem(name='飲料生産', sense=pulp.LpMaximize)

x1 = pulp.LpVariable('x1', lowBound=0, cat='Continuous')
x2 = pulp.LpVariable('x2', lowBound=0, cat='Continuous')

model += 3000 * x1 + 2000 * x2          # 目的関数
model += 2 * x1 + 5 * x2 <= 500        # リンゴ制約
model += 4 * x1 + 2 * x2 <= 300        # オレンジ制約

model.solve()
print('Status:', pulp.LpStatus[model.status])
print('x1 =', x1.value(), 'x2 =', x2.value())
# → Status: Optimal, x1 = 31.25, x2 = 87.5
```

モデラー作成 6 ステップ

- 1. 問題定義
 - LpProblem(name, sense)
- 2. 決定変数
 - LpVariable(name, bounds, cat)
- 3. 目的関数
 - model += expr
- 4. 制約式
 - model += expr <= rhs
- 5. 求解
 - model.solve()
- 6. 値取り出し
 - x.value()
- ML の fit→predict と相似形

整数計画 (IP) と 0-1 整数計画

④ 整数計画問題

決定変数が整数値のみ。特に $\{0, 1\}$ に限定されたものを 0-1 IP と呼ぶ

種類と難しさ

- 0-1 IP: 選択する/しない (binary)
 - → ナップサック、施設配置の「契約 or NOT」
 -
- 線形 IP: 整数 + 線形目的・制約
 - → ビンパッキングなど
 -
- 非線形 IP: 二乗誤差や $\log$ を含む
 - → PuLP + Cbc では解けない
 -
- ★MaxCut / Ising は二次形の 0-1 IP
 - 決定変数 $x_i \in \{0, 1\}$ が「集合 A か B か」
 - 目的 = $\sum (x_i - x_j)^2$ over edges (i, j)

ナップサック問題の定式化

- 決定変数: $x_i \in \{0, 1\}$ 商品 i を入れるか
-
- 目的: maximize $\sum_i v_i x_i$
-
- 制約: $\sum_i w_i x_i \leq C$ (容量)
-
- 実装は LP とほぼ同じ:
 - `pulp.LpVariable.dicts('x', I, cat='Binary')`
 -
- ここで集合 (I) と総和記号 ($\sum$) で書く
 - → 入力サイズに依らない汎用コード
 -
- MaxCut も「集合 + $\sum$ + 0-1 変数」で全部書ける

混合整数計画 (MIP) — 施設配置

⑤ MIP 定式化

実数決定変数と整数決定変数が同居 — 現実問題の多くがここに該当

問題: 倉庫契約と輸送量

- 5 つの倉庫から 4 店舗へ製品を配送する
- $y_i \in \{0, 1\}$ 倉庫 i を契約するか (整数!)
- $x_{ij} \in \mathbb{R} \geq 0$ i から j への輸送量 (実数!)
-
- 目的: $\min \sum_i f_i y_i + \sum_{i,j} c_{ij} x_{ij}$
- (固定費 + 輸送費)
-
- 制約:
- $\sum_j x_{ij} \leq M_i y_i$ 容量(big-M トリック)
- $\sum_i x_{ij} = d_j$ 店舗 j の需要充足
-
- $\rightarrow$ 整数 + 実数の両方を扱う = MIP

big-M トリックの読み方

- $\sum_j x_{ij} \leq M_i y_i$
-
- $y_i = 0$ のとき右辺 = 0 $\rightarrow$ 倉庫 i は 1 単位も出せない
- $y_i = 1$ のとき右辺 = M_i $\rightarrow$ 倉庫 i の容量分だけ出せる
-
- = 「整数変数で連続変数のオン/オフを切る」
-
- ★Ising / MaxCut でも同じ構造が随所に出る
- $\sigma_i = \pm 1$ で結合項 $J_{ij} \sigma_i \sigma_j$ をオン/オフ
- 「離散変数で連続物理量の挙動を切替える」
-
- MIP の感覚を持っておくと
- 自分の問題を MILP ソルバーに投げる選択肢が生まれる

ハード制約 → ソフト制約

実行可能性が壊れたときの逃がし方

等式 $\Sigma = \text{AvgPoints}$ のように厳密に守らせると実行不能になりがち

ハード制約だと壊れる例

- 受講生のスキル合計を 8 グループに均等配分したい
- $\Sigma_i \text{skill}_i x_{i,g} = \text{AvgPoints} \quad \forall g$
- → skill が整数で、AvgPoints が割り切れなければ
- 実行可能解が存在しない
- 「絶対こうしたい」を絶対にすると、
- ちょっとした入力で全部壊れる

ソフト化: 偏差を最小化

- $\Sigma_i \text{skill}_i x_{i,g} + u_g - o_g = \text{AvgPoints}$
- $u_g, o_g \geq 0$
- 目的に偏差ペナルティを足す:
• minimize $\Sigma_g (u_g + o_g)$
- ★研究室文脈:
• CIM/CAC のロス関数 = エネルギー + ペナルティ
• 「ペナルティ係数の重み」が実装上の自由度
• ハード制約を直接書くより、ペナルティ化で
• 最適化問題に乗せ直すアプローチが多い

Part III

研究室の研究との接続

MaxCut を 0-1 IP として書く

研究テーマを定式化の言葉で

G-set / G22 (N=2000) — MILP ソルバーの「典型問題」と同じ枠組みで書ける

(1) ナイーブな 0-1 IP

- 決定変数: $x_i \in \{0, 1\}$ 頂点 i を集合 A か B かで分ける
-
- 辺 (i, j) がカットされる $\Leftrightarrow x_i \neq x_j$
- $\Leftrightarrow x_i + x_j - 2x_i x_j = 1$
-
- 目的:
- maximize $\sum_{(i,j) \in E} (x_i + x_j - 2x_i x_j)$
-
- $\rightarrow$ 2 次項が出る $\Rightarrow$ QUBO 形式
- $\rightarrow$ Cbc では解けない (Gurobi なら可)

(2) 線形化 IP (本物の MILP)

- 新変数 $y_{ij} \in \{0, 1\}$ を導入 — $y_{ij} = (x_i \text{ XOR } x_j)$
-
- maximize $\sum_{(i,j) \in E} y_{ij}$
-
- subject to:
- $y_{ij} \leq x_i + x_j$
- $y_{ij} \leq 2 - x_i - x_j$
- $y_{ij} \geq x_i - x_j$
- $y_{ij} \geq x_j - x_i$
-
- ★MILP ソルバーで解ける形に変換可能
- だが N=2000 で辺 19990 本では現実時間で解けない

MILP ソルバー vs CIM / CAC

アプローチの住み分け

	MILP ソルバー (Gurobi/Cbc)	CIM / CAC / SA / SB
定式化	0-1 IP / QUBO の線形化	Ising ハミルトニアン $H = -\sum J_{ij} \sigma_i \sigma_j$
決定変数	$x_i \in \{0, 1\}$	$\sigma_i \in \{-1, +1\}$ (光振幅 c_i の符号)
保証	最適性ギャップを数値で出せる	近似解。最適性証明なし
小規模	$N \leq 100$ なら数秒で厳密最適解	オーバーキル
G-set 規模	$N=2000$ では時間切れになる場合あり	数百 ms で best_cut ≈ 13320 (G22)
パラメータ	ほぼなし (gap, time limit 程度)	kappa, gamma, dP/round, 損失 dB …多数
役割	「正解はこれ」と保証する基準	規模を上げたときの実用的な解法

→ どちらか一方ではなく、「小規模は MILP で答合わせ、大規模は CIM」が現実的

Optuna も「最適化問題」だった

連続・ブラックボックス最適化として

BrainPad の分類でいうと「連続 × 非線形 × ブラックボックス目的関数」に該当

Optuna の気持ちを LP/IP の言葉で

- 決定変数 (連続):
 - L, gamma, loss_dB, dP_per_round, coupling
 -
- 目的関数 (ブラックボックス):
 - $f(\text{params}) = 20$ 試行平均の best_cut
 -
- 制約条件:
 - 各パラの探索範囲 (suggest_float の min/max)
 -
- → 「数式で目的関数が書けない」点だけが LP と違う
- → TPE 等のサンプラーで賢く点を打って探す

今週やった num_rounds スイープの位置づけ

- 外側ループ (人がやる): $\text{num_rounds} \in \{30, 300, 3k, 10k\}$
- 内側ループ (Optuna): 5 パラを TPE で探索
 - → 計算予算 = (rounds × n_trials) ほぼ一定
 -
- 得られた知見:
 - 30 rounds: 論文値 10337 → Optuna 12886 (+2549)
 - 3000+ rounds: 既に飽和し改善幅 +20 程度
 -
- = 数理最適化の枠組みで「最適化を最適化」している
- (メタ最適化)

今後 研究室でこう活かす

学んだことの再翻訳

- 自分の問題を「3 要素」で書く癖
 - CIM 研究でも、評価関数を変えるときは『目的・変数・制約』に分解する
 - ペナルティ係数の議論は「ソフト制約のチューニング」と同じ
-
- 典型問題のカatalogを持つ
 - ナップサック / 施設配置 / 割当 / スケジューリングは応用範囲が広い
 - MaxCut の派生(重み付き / k-cut / 制約付き)を IP として書けるようにする
-
- 小規模ベンチマークで答え合わせをする
 - $N \leq 100$ の MaxCut なら Gurobi で厳密最適を求めて CIM と比較できる
 - 「13321 が本当の最適か」を MILP ソルバーで証明する逆方向の研究も可能
-
- メタ最適化を意識的に設計する
 - Optuna 探索空間・予算配分・warm start は「最適化問題の設計問題」と捉える
 - BrainPad の「ハード→ソフト制約」の発想で、Optuna の探索を頑健にできる

まとめ

持ち帰る 3 つのメッセージ

①

数理最適化 = 目的関数 / 決定変数 / 制約 の 3 要素で世界を書き直す技術

自分の研究も「何を最大化したいか」を言語化する練習になる

②

LP / IP / MIP は「問題のタイプ」のラベル。MaxCut は 0-1 IP の典型

CIM はその「フルスクラッチ・問題固有」アプローチ

③

ソルバーは選ぶもの。汎用ソルバー / CIM / Optuna は同じ最適化の家族

問題規模と保証要件で住み分ければよい

参考文献 / 今後の学習リスト

⑧ 今後学ぶと良いこと

典型問題を学ぶ書籍

- 「Python ではじめる数理最適化」
- (BrainPad DS 新卒指定書籍)
- 実務寄り、モデリング多数
-
- 「あたらしい数理最適化」
- 典型問題のコツが詰まっている
-
- Williams 「Model Building in Mathematical Programming」
- MIP モデリングの聖典
-
- 梅谷俊治 「しっかり学ぶ数理最適化」
- 理論寄り (BrainPad 資料の引用元)

次のステップ (研究との結合)

- ヒューリスティクスを学ぶ
- AtCoder Heuristic Contest (AHC)
- 焼きなまし・ビームサーチ etc.
- → CIM/CAC を SA と並べる審美眼が育つ
-
- 他ソルバーに触れる
- OR-Tools CP-SAT (論理制約に強い)
- Gurobi (有償だが学術ライセンス可)
-
- Web アプリ化
- 研究結果をデモにしたいときに必要
- Streamlit + PuLP の組合せが簡単

原典: BrainPad 『数理最適化研修』 (社内公開資料, 2025) を研究室文脈で再構成 — 内容の責任は本発表者